Blockchain : Experiment - 7

Aim:

Introduction to Truffle, Establish local blockchain using Truffle and deployment of Smart Contract.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connect Ganache with Metamask and Remix IDE:

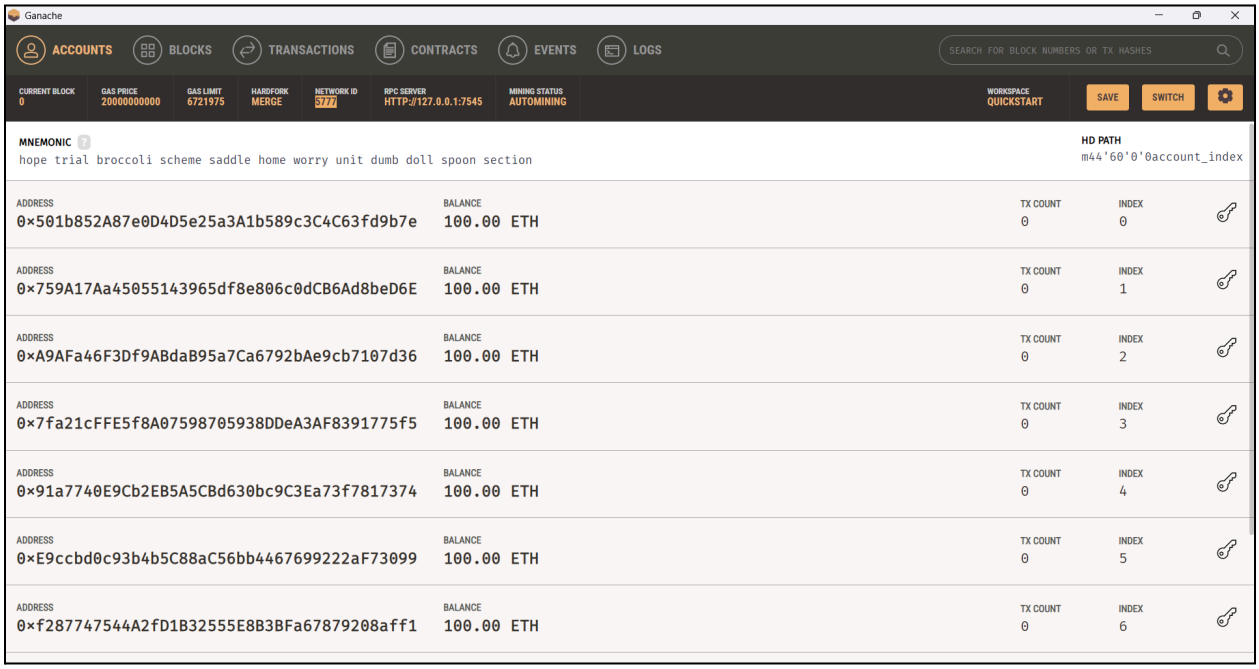
1. Install Ganache:
    a. Download and install Ganache from the Truffle Suite website.
    b. Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
    a. Open Metamask and switch to a Custom RPC Network.
    b. Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
    c. Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
    a. Open Remix IDE in the browser.
4. Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
    a. Remix will automatically connect to the Metamask account (linked with Ganache).
5. Create a Simple Solidity Smart Contract:
    a. Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).
6. Compile and Deploy the Contract:
    a. Compile using Remix.
    b. Deploy via the Metamask account connected to Ganache.
    c. Approve the transaction in Metamask.
7. Check Transaction Details on Ganache:
    a. View mined transactions, block details, and balances in the Ganache interface.
8. Interact with the Smart Contract:
    a. Use Remix to call contract functions (set() or get()).
    b. Observe the state changes and corresponding transactions on Ganache.

Code:

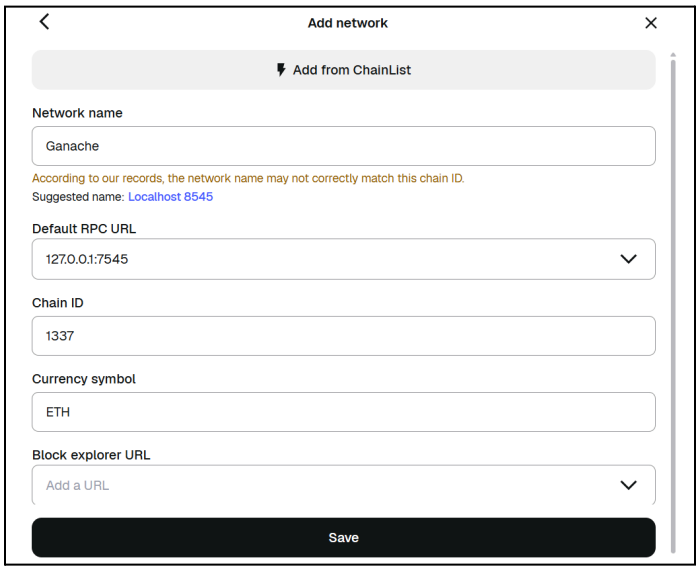
```
pragma solidity ^0.8.0;
contract DrugSupplyChain {
  enum Stage { Manufactured, WithDistributor,
WithRetailer, Sold }
  struct Drug {
    uint256 drugID;
    string name;
    uint256 expiryDate;
    address manufacturer;
    address distributor;
```

```
    address retailer;
    address customer;
    Stage stage;
    uint256 lastUpdated;
  }
  mapping(uint256 => Drug) public drugs;
  event DrugManufactured(uint256 drugID,
string name, address manufacturer);
  event SentToDistributor(uint256 drugID,
address distributor);
  event SentToRetailer(uint256 drugID, address
retailer);
  event SoldToCustomer(uint256 drugID,
address customer);
  function manufactureDrug(uint256 _drugID,
string memory _name, uint256 _expiryDate)
public {
    require(_drugID != 0, "Invalid drug ID");
    require(drugs[_drugID].drugID == 0, "Drug
already exists");
    drugs[_drugID] = Drug({
      drugID: _drugID,
      name: _name,
      expiryDate: _expiryDate,
      manufacturer: msg.sender,
      distributor: address(0),
      retailer: address(0),
      customer: address(0),
      stage: Stage.Manufactured,
      lastUpdated: block.timestamp
    });
    emit DrugManufactured(_drugID, _name,
msg.sender);
  }
  function sendToDistributor(uint256 _drugID,
address _distributor) public {
    Drug storage d = drugs[_drugID];
    require(d.drugID != 0, "Drug does not
exist");
    require(msg.sender == d.manufacturer,
"Only manufacturer allowed");
    require(d.stage == Stage.Manufactured,
"Wrong stage");
    d.distributor = _distributor;
    d.stage = Stage.WithDistributor;
    d.lastUpdated = block.timestamp;
    emit SentToDistributor(_drugID,
_distributor);
  }
  function sendToRetailer(uint256 _drugID,
address _retailer) public {
    Drug storage d = drugs[_drugID];
    require(d.drugID != 0, "Drug does not
exist");
    require(msg.sender == d.distributor, "Only
distributor allowed");
    require(d.stage == Stage.WithDistributor,
"Wrong stage");
    d.retailer = _retailer;
    d.stage = Stage.WithRetailer;
    d.lastUpdated = block.timestamp;
    emit SentToRetailer(_drugID, _retailer);
  }
  function sellToCustomer(uint256 _drugID,
address _customer) public {
    Drug storage d = drugs[_drugID];
    require(d.drugID != 0, "Drug does not
exist");
    require(msg.sender == d.retailer, "Only
retailer allowed");
    require(d.stage == Stage.WithRetailer,
"Wrong stage");
    require(block.timestamp < d.expiryDate,
"Drug expired");
    d.customer = _customer;
    d.stage = Stage.Sold;
    d.lastUpdated = block.timestamp;
    emit SoldToCustomer(_drugID, _customer);
  }
  function getDrug(uint256 _drugID) public
view returns (Drug memory) {
    require(drugs[_drugID].drugID != 0, "Drug
does not exist");
    return drugs[_drugID];
  }
}
```

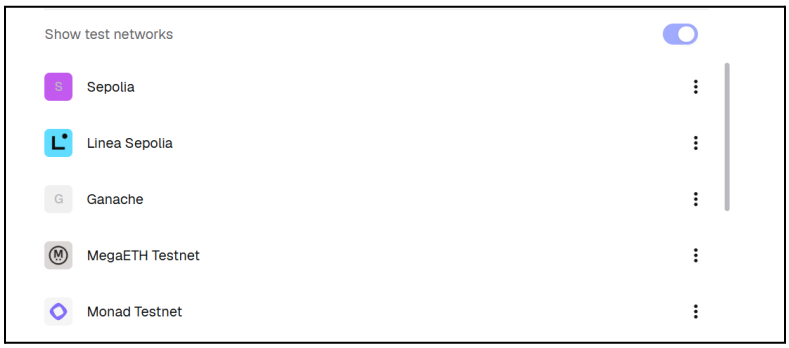
Output:



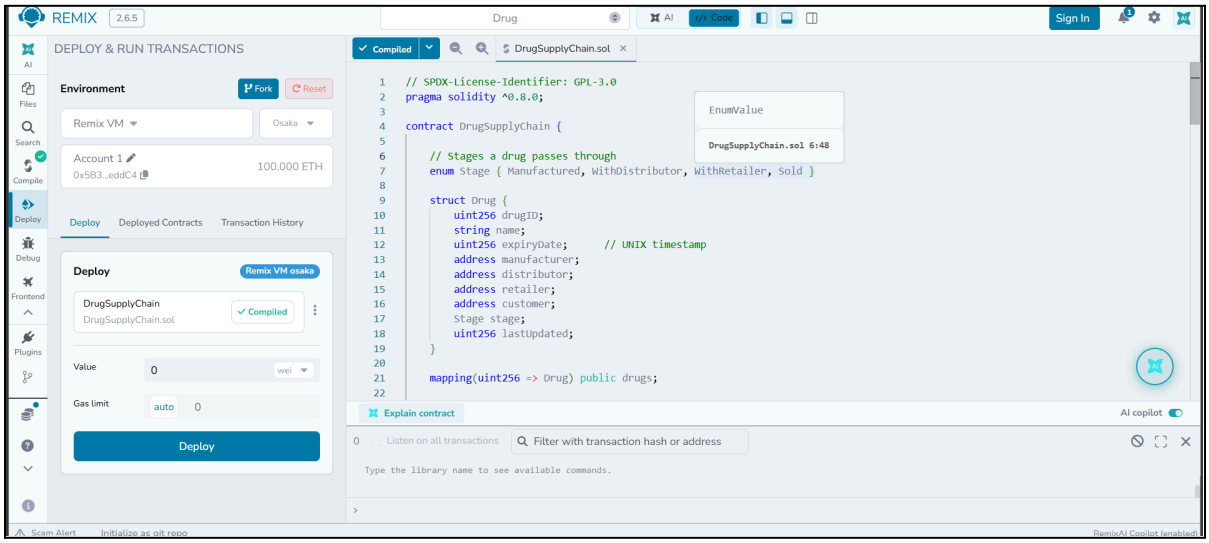
Connect Ganache with MetaMask:



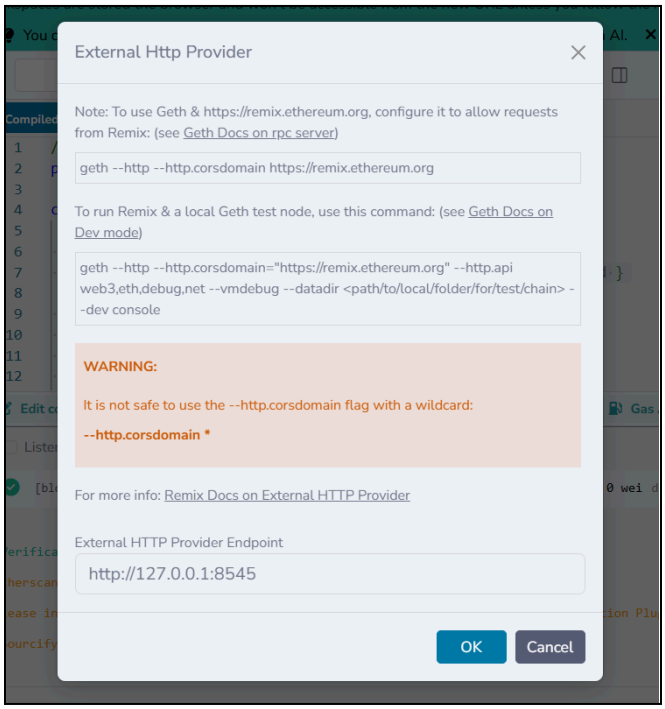
You can see the created network:



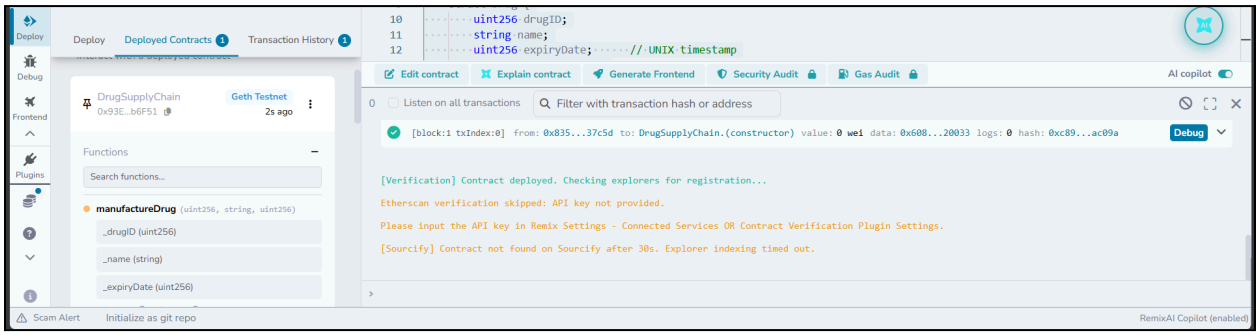
Write the smart contract code:

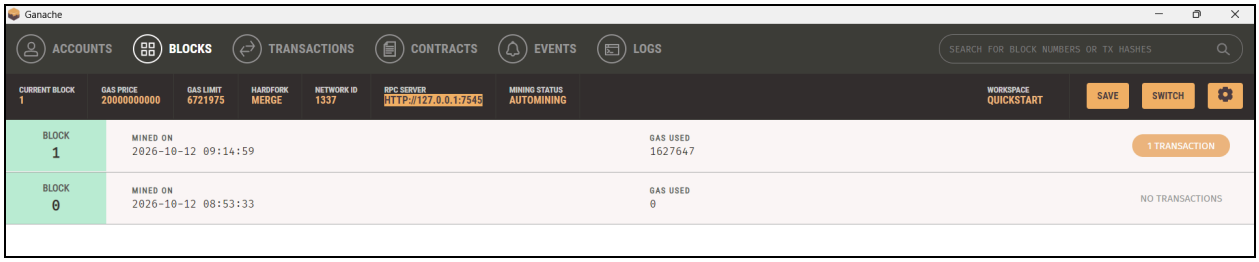


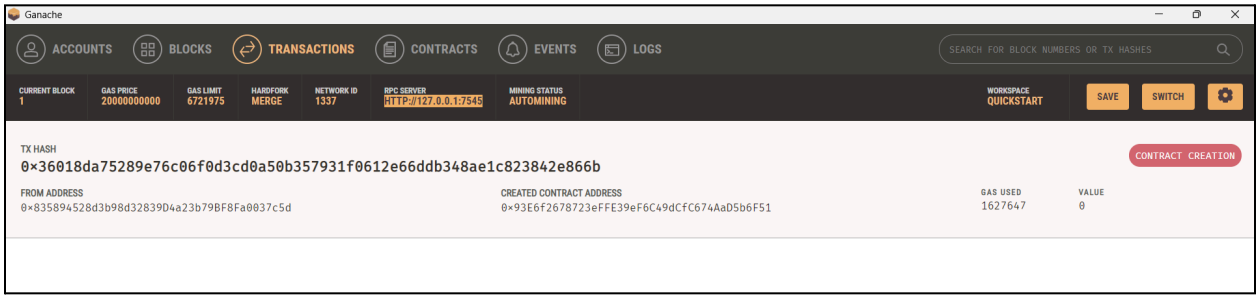
Connect RemixIDE with Ganache:



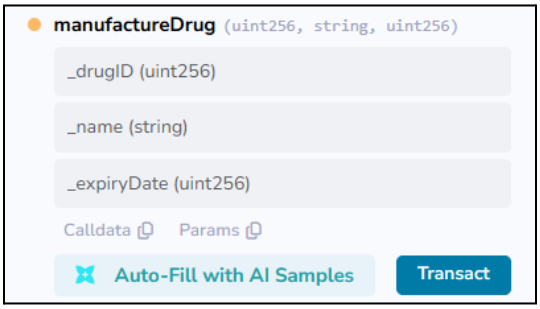
Smart Contract Deployed:

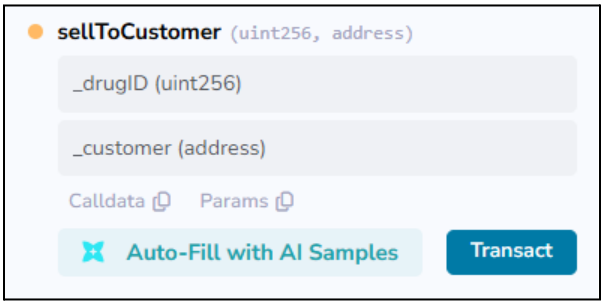


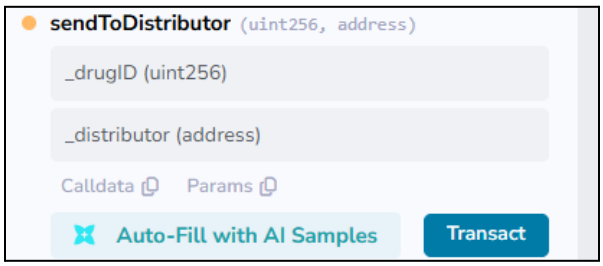


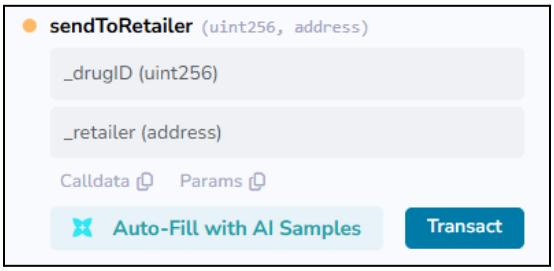


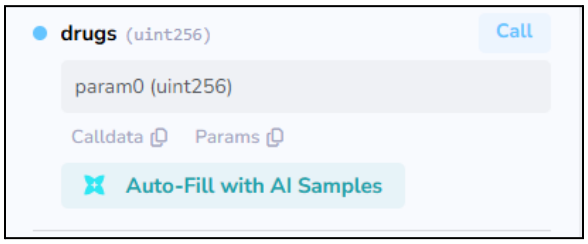
Functions of smart contract:













Explanation of functions:

1. manufactureDrug(): Registers a new drug with a unique ID, name, and expiry date. The caller is recorded as the manufacturer, and the initial stage is set to Manufactured.
2. sendToDistributor(): Transfers responsibility for a drug from the manufacturer to a distributor. It verifies the manufacturer's identity and updates the drug's stage to WithDistributor.
3. sendToRetailer(): Transfers responsibility from the distributor to a retailer. It verifies the distributor's identity and updates the drug's stage to WithRetailer.

4. sellToCustomer(): Records the sale of a drug to a customer. It verifies the retailer's identity, checks that the drug has not expired, and updates the stage to Sold.
5. getDrug(): Retrieves the complete details of a drug, including its ID, name, expiry date, manufacturer, distributor, retailer, customer, current stage, and last updated timestamp.
6. drugs(): An automatically generated getter function that retrieves the stored details of a drug using its ID.

Conclusion:

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with Metamask and Remix, developers can deploy, test, and debug smart contracts efficiently before moving to a public network.